

CMPE 220 — System Software

Spring 2026

Assignment 2

Program Layout and Execution

Team members

Faiq Malik

Date: May 15, 2026

GitHub Repository

The complete source code, any built artifacts or example binaries included in the repository, demonstration videos, and documentation are maintained at:

`https://github.com/Faiq2001/software\_cpu\_design-main`

The repository root contains `cpu16_core.cpp` (CPU16 assembler and emulator), `factorial.c` (recursive factorial with `main` driver), prebuilt images such as `fact.bin` and `fib.bin` when provided, demonstration media under `demos/` (including `demos/Demo_Recursion_Memory.mp4`), diagrams under `figures/` when present, and this report under `reports/`.

Download, Compile, and Run

Download

Clone the repository with Git, or download a ZIP archive from the GitHub page above and extract it to a working directory on macOS or Linux.

Requirements

- C++ compiler with C++17 support (**g++** recommended).
- C compiler with C11 support (**gcc** recommended) for the factorial program.
- Terminal access.

Compile

From the repository root directory, build CPU16:

```
g++ -std=c++17 -O2 -o cpu16 cpu16_core.cpp
```

This produces the executable **cpu16**, which implements assembler mode (**asm**), bare-memory emulator mode (**emu**), and assemble-and-run mode (**run**).

Build the C recursive factorial program:

```
gcc -std=c11 -O2 -o factorial factorial.c
```

Run

Run the C program (enter a non-negative integer when prompted):

```
./factorial
```

Emulate a factorial program image with memory dump (when `fact.bin` is present in the project directory, from assembly or distributed with the repository):

```
./cpu16 emu fact.bin --base 0x0000 --pc 0x0000 --dump 0x0000 0x01FF
```

This loads the binary into the simulated 64K address space at base `0x0000`, starts at program counter `0x0000`, and prints a hex dump of `0x0000–0x01FF` so code, data, and stack regions are visible while recursive calls use `CALL/RET`.

Typical commands when example assembly sources are under `examples/`:

```
./cpu16 run examples/hello.asm
./cpu16 run examples/timer.asm
./cpu16 asm examples/fib.asm -o fib.bin
./cpu16 emu fib.bin --base 0x0000 --pc 0x0100 --dump 0x0100 0x0140
```

The Fibonacci listing uses `.org 0x0100`; when emulating `fib.bin`, set the program counter to `0x0100` as in the command above. If `fib.bin` lives in the repository root instead of beside the assembly file, use the same arguments with that path, for example:

```
./cpu16 emu fib.bin --base 0x0000 --pc 0x0100 --dump 0x0100 0x0140
```

To assemble factorial from source when `examples/fact.asm` is available, then load and dump memory:

```
./cpu16 asm examples/fact.asm -o fact.bin
./cpu16 emu fact.bin --base 0x0000 --pc 0x0000 --dump 0x0000 0x01FF
```

Video

The repository includes `demos/Demo_Recursion_Memory.mp4` (function calls, recursion in C and on CPU16, assembly context, and memory layout using emulator dumps).

Team Member Contributions

Faiq Malik

Contributions include:

- **Recursive C program (factorial.c):** Implemented a recursive factorial function with a base case for $n \leq 1$ and a recursive multiply-and-call step; wrote the `main` driver that reads user input, validates non-negative input, invokes the recursive function, and prints the result. Added comments tying the program to the stack-layout comparison with the simulated CPU.
- **CPU16 tool chain (cpu16_core.cpp):** For this assignment, used the software CPU (assembler and emulator: `asm`, `emu`, `run`) to build and run factorial images, set base address and program counter for `emu`, and capture `--dump` output to relate code, data, and stack to physical addresses during recursive calls.
- **Program layout demonstration:** Produced or maintained assembled images (for example `fact.bin`) and documented how to use `emu` with `--dump` to inspect memory layout, illustrating how function calls and recursion map to concrete addresses on CPU16.
- **Repository organization:** Structured the repository with a clear layout: sources at the root, `reports/` for course report PDFs, `demos/` for videos, `figures/` for diagrams, plus binaries and build outputs as needed.
- **Video deliverables:** Recorded `demos/Demo.mp4` (CPU16 overview) and `demos/Demo_Recursion_Memory.mp4` (function calls, recursion execution, assembly-level discussion, and memory layout).
- **Deliverables:** Repository layout, grader-facing build and run documentation (including this report), demonstration videos as required, supporting assets as required, and this report submitted as PDF for the course.